
Deep Learning for High-Frequency Bitcoin Limit Order Book Forecasting

Abstract

Sub second order book dynamics on Bitcoin perpetual futures push the limits of what convolutional and attention based deep learning models can capture from raw limit order book (LOB) data. We compare five short horizon mid-price classifiers on a five day BTCUSDT.P sample of approximately 1.7 million 250 ms snapshots. The five models are an Order Flow Imbalance (OFI) baseline, a plain LSTM, DeepLOB, LiT and our proposed *Attentive DeepLOB*¹, which augments DeepLOB with two axial self-attention blocks. A central focus of this work is to determine whether these architectures generalise beyond the event sampled datasets that have been tested in prior work to an arbitrary five day window. All five models share a common preprocessing, labelling and day based chronological split. We evaluate accuracy, balanced accuracy and macro and weighted F1 across 1 s, 2 s and 5 s horizons. The OFI baseline remains a non-trivial benchmark above the 33% chance level and the deep models substantially outperform it. Attentive DeepLOB obtains the best accuracy at 1 s (68.5%) and 5 s, DeepLOB leads at 2 s and the original DeepLOB exhibits a small (4-6 percentage point) train-validation gap concentrated in the first few epochs. These findings suggest deep LOB forecasting is primarily a robustness problem and indicate that combining convolutional spatial inductive bias with global axial attention can outperform either component alone on noisy crypto LOB data.

1. Introduction

A limit order book (LOB) is a list of all outstanding buy and sell orders for an asset, organised by price. Buy orders, called bids, sit at prices below the current trading price and sell orders, called asks, sit above. The best bid is the highest current buy price and the best ask the lowest current sell price. The midpoint between the two is the mid-price and the difference is the spread, which measures the cost of crossing

the book to take liquidity. Forecasting whether the mid-price will move up, move down or remain approximately unchanged over the next short horizon is directly relevant to market makers, algorithmic traders and risk managers.

Cryptocurrency markets, and in particular Bitcoin perpetual futures, offer an especially demanding setting for this forecasting problem. Perpetual futures track the underlying asset price but unlike standard futures do not expire. Centralised crypto platforms operate continuously, attract globally distributed order flow and exhibit very high trading activity. At sub second timescales, price movements are not driven by macroeconomic news but by microstructure dynamics including the immediate interaction of orders, cancellations, liquidity and queue imbalances within the order book (Cont et al., 2014). This makes the raw LOB snapshot a natural input for short horizon forecasting models. The task is conceptually challenging because labels are noisy, classes are heavily imbalanced at sub second horizons and naive baselines can already achieve high raw accuracy by predicting the majority class, so meaningful evaluation requires balanced metrics and theoretically grounded baselines.

Existing baselines. Recent advances in deep learning have produced architectures specifically designed for LOB data. Zhang et al. (2019) introduced DeepLOB, which exploits the spatial structure of the LOB snapshot via convolutional layers and an Inception module before capturing temporal dynamics with an LSTM, an architecture we describe in detail in Section 4.3. Transformer-based alternatives have followed, replacing locality-biased convolutional feature extraction with global self-attention (Berti & Kasneci, 2025; Xiao et al., 2025). Together with the OFI baseline of Cont et al. (2014) and a plain LSTM that strips out any spatial inductive bias, these four models form the established baselines against which we measure architectural progress.

Sampling regimes and our contribution. The LiT paper directly compares its model against DeepLOB on Binance Bitcoin perpetual futures, but does so under event-based sampling. A new snapshot is recorded only when an order book event occurs, following Ntakaris et al. (2018). Event-based sampling concentrates the data on moments where something is happening, but it changes the nature of the prediction task in ways that matter for deployment. The model never has to handle quiet stretches where the book is stable, and the time interval between consecutive samples becomes

¹Referred to as DeepLOB v2 in the code and plots.

a function of market activity that the model can implicitly exploit. Furthermore, prediction horizons defined in milliseconds (such as the 300-500 ms windows used by [Xiao et al. \(2025\)](#)) actually correspond to “the next event after this many milliseconds have elapsed”, which is not the same target as predicting the price one second from now in clock time. [Zhang et al. \(2019\)](#) themselves describe such datasets as “far too short, downsampled and taken from a less liquid market,” and [Briola et al. \(2025\)](#) document that several state-of-the-art models overfit these benchmarks and lose substantial performance on more realistic data. Sampling also affects the empirical class distribution. Under uniform sampling, long quiet stretches are over-represented and the smoothed-label scheme of [Zhang et al. \(2019\)](#) assigns most of them to the stationary class, whereas event-sampling concentrates observations on directional moves and so produces a more balanced label distribution.

We re-evaluate DeepLOB and LiT on the same uniform 250 ms-sampled BTCUSDT.P stream, with rolling z-score normalisation. Both flagship models therefore face the same task that a clock-driven trader actually faces, every quiet stretch is preserved and prediction horizons in seconds correspond to clock time. After comparing these four established baselines under this shared setting, we use the observed strengths and weaknesses to motivate *Attentive DeepLOB*, an extension that retains DeepLOB’s price-volume convolutional front end and inserts two axial self-attention blocks before the LSTM. The intuition is that DeepLOB’s strong but local convolutional bias may be too rigid for noisy crypto data, and that adding global attention without abandoning the convolutional front end could combine the strengths of both architectures. All five models operate on the same 40-feature LOB input across three horizons ($k \in \{4, 8, 20\}$ snapshots, equivalent to 1 s, 2 s and 5 s).

Findings. The OFI baseline reaches 40-43% accuracy across the three horizons, well above the 33% chance level. The deep sequence models meaningfully outperform OFI. At 1 s, Attentive DeepLOB achieves 68.5% accuracy and 68.5% macro F1, the best of any model and metric on that horizon. DeepLOB leads at 2 s and Attentive DeepLOB leads again at 5 s. The original DeepLOB exhibits a small (3.6-6.7 percentage point) train-validation accuracy gap that is concentrated in the first epochs, where the model overshoots before early stopping engages. These results suggest that under a controlled, clock-time evaluation on a noisy crypto LOB, combining convolutional spatial inductive bias with global axial attention modestly but consistently outperforms either component alone.

2. Related Work

Early approaches to LOB mid-price prediction relied on handcrafted features fed into logistic regression or support

vector machines ([Kercheval & Zhang, 2015](#)). [Ntakaris et al. \(2018\)](#) later released the FI-2010 benchmark dataset. [Sirignano & Cont \(2019\)](#) demonstrated that deep neural networks discover universal price formation features across thousands of US equities without feature engineering. Convolutional approaches followed quickly. [Tsantekidis et al. \(2017\)](#) pioneered convolutional networks for LOB data and [Zhang et al. \(2019\)](#) significantly improved on this with DeepLOB. More recent work has adapted attention mechanisms to this setting. [Berti & Kasneci \(2025\)](#) introduced TLOB and its dual-attention separation of temporal and spatial heads, [Kisiel & Gorse \(2022\)](#) proposed axial attention along the price and time axes to reduce quadratic cost, and [Xiao et al. \(2025\)](#) introduced LiT with structured patches that respect the bid and ask boundary.

[Briola et al. \(2025\)](#) introduced LOBFrame, linking model predictability to microstructural properties such as tick size and spread, and showed that classical machine learning metrics can be misleading as practical trading signals. [Wang \(2025\)](#) found that simpler models can match deep architectures on crypto LOB data when preprocessing is careful, emphasising that better feature engineering often matters more than additional model complexity. [Cont et al. \(2014\)](#) provide the theory-driven counterpoint with Order Flow Imbalance, which is derived from first principles of order book dynamics and remains the standard linear benchmark in the microstructure literature.

3. Data

Dataset. We use a contiguous sample of Bitcoin perpetual futures (BTCUSDT.P) LOB data sampled at 250 ms intervals, covering 9-15 January 2023. After discarding partial days at the boundaries, the sample retains five full trading days (10-14 January 2023) comprising approximately 1,722,931 snapshots. Each snapshot records the top 10 bid and top 10 ask price levels with their corresponding volumes, yielding 40 features per timestamp. In our sample the spread is at the 0.1 USDT minimum tick for the vast majority of observations (median spread 0.1 USDT), confirming that BTCUSDT.P is highly liquid on this exchange. Mid-prices range from approximately 17,118 to 21,516 USDT over the window. The mid-price and spread time series are shown in Figure 1 (See Appendix).

Feature ordering. We reorder columns to the DeepLOB format $\{P_i^{\text{ask}}, V_i^{\text{ask}}, P_i^{\text{bid}}, V_i^{\text{bid}}\}_{i=1}^{10}$ following [Zhang et al. \(2019\)](#). This ordering is necessary because DeepLOB’s first convolution applies a (1×2) filter with stride 2 that combines two adjacent features into a single output. Pairing each price with its own volume at the same level, rather than mixing features across levels or across the bid and ask side of the book, preserves the price-volume coupling that the architecture is designed to exploit.

Normalisation. Raw LOB features have very different magnitudes, with prices in the tens of thousands of USDT and volumes typically below ten BTC. Without normalisation, gradient updates would be dominated by the high magnitude price features, and standard weight initialisation schemes that assume zero-mean unit variance inputs would fail. We standardise each feature using a rolling z-score procedure computed per feature, using up to the previous five available dates as the normalisation window. This produces 40 input channels of comparable scale while avoiding any forward looking information leakage. The procedure follows the rolling window convention used by Tsantekidis et al. (2017) and Zhang et al. (2019) but applied per day rather than per minute, because BTC volatility is much more stable on a daily timescale than on the minute timescale of equity markets.

Mid-price and labelling. The mid-price at time t is $p_t^{\text{mid}} = (P_1^{\text{ask}}(t) + P_1^{\text{bid}}(t))/2$. We use the smoothed labelling scheme from the original DeepLOB paper (Zhang et al., 2019). For horizon k in 250 ms snapshots, define the smoothed backward and forward averages

$$m^-(t) = \frac{1}{k+1} \sum_{i=0}^k p_{t-i}^{\text{mid}}, \quad m^+(t) = \frac{1}{k} \sum_{i=1}^k p_{t+i}^{\text{mid}}. \quad (1)$$

The normalised price change is $\ell_t = (m^+(t) - m^-(t))/m^-(t)$ and the label is

$$y_t = \begin{cases} 2 \text{ (up)} & \text{if } \ell_t > \alpha, \\ 1 \text{ (stationary)} & \text{if } |\ell_t| \leq \alpha, \\ 0 \text{ (down)} & \text{if } \ell_t < -\alpha, \end{cases} \quad (2)$$

where $\alpha > 0$ is a threshold tuned per horizon. Smoothing with $m^-(t)$ reduces label noise from microstructure bounce, the rapid alternation of trades between bid and ask within a single tick that creates mechanically alternating mid-prices not driven by any fundamental price change. Smoothing with $m^+(t)$ makes the label representative of the average future price level rather than a single noisy tick.

We tune α per horizon via bisection search to target a 33% stationary fraction on the training set, so that the resulting class distribution is approximately balanced in training and chance accuracy is well-defined at one third. Table 1 (See Appendix) reports the resulting training-set class distributions and test-set class counts per horizon. The training distribution is more imbalanced than the 33%/33%/33% target at the 1 s and 2 s horizons, because the bisection search struggles to reach the target when the training window volatility is dominated by quiet stretches. At very small α almost every update is classified as directional, and at moderate α the stationary class exceeds the target. By the 5 s horizon the natural distribution of smoothed returns has spread out enough that the bisection converges close to the 33%

target. The test class proportions differ from the training distribution because mid-price drift over the five-day window changes the balance of directional moves, and we report this drift transparently instead of retuning α on test data.

Train, validation and test split. A strict day based chronological split was adopted, with training on the first three full days (10-12 January 2023), validation on the fourth (13 January) and test on the fifth (14 January). Partial days at the file boundaries (9 January, 1.7 h; 15 January, 22.7 h) are discarded so that each split contains a complete intra-day volatility cycle and is not biased by an unrepresentative slice of the day. The resulting row counts are approximately 1,033,687 training rows, 344,628 validation rows and 344,616 test rows. The chronological ordering prevents data leakage. Sequence construction ensures that neither the lookback window $[t - T + 1, \dots, t]$ nor the forecast horizon $[t + 1, \dots, t + k]$ crosses a split boundary.

Table 1. Training-set class distributions after training-slice α calibration, and test-set class counts. Test support is the number of valid sequence-label pairs after enforcing no split-boundary crossing.

Horizon		Down	Stat.	Up
1 s	Train	18.1%	64.3%	17.6%
	Test	36.7%	26.4%	36.9%
2 s	Train	24.5%	51.5%	24.0%
	Test	43.5%	12.7%	43.8%
5 s	Train	32.5%	35.3%	32.2%
	Test	47.4%	4.3%	48.3%

4. Model Architectures

All five models share the same lookback window of $T = 50$ consecutive normalised LOB snapshots, corresponding to 12.5 seconds of market history. Zhang et al. (2019) used $T = 100$. We reduced this to $T = 50$ to keep the per-batch tensor size manageable on a single GPU while retaining a window long enough to span the order flow imbalance autocorrelation timescale documented by Cont et al. (2014). For the neural models the input is a 50×40 matrix and the output is a probability distribution over three classes (down, stationary, up) via a softmax layer. Models are trained independently for each forecast horizon. All deep models use class-balanced sample weights during training to account for residual class imbalance in the training set.

4.1. Order Flow Imbalance (OFI)

The OFI model (Cont et al., 2014) is a theory-driven linear baseline. At each level n and timestep t it tracks the net

change in queue size on the bid side relative to the ask side:

$$W_n(t) = q_n^b(t)\mathbb{I}[p_n^b(t) \geq p_n^b(t-1)] - q_n^b(t-1)\mathbb{I}[p_n^b(t) \leq p_n^b(t-1)], \quad (3)$$

$$V_n(t) = q_n^a(t)\mathbb{I}[p_n^a(t) \leq p_n^a(t-1)] - q_n^a(t-1)\mathbb{I}[p_n^a(t) \geq p_n^a(t-1)], \quad (4)$$

$$e_n(t) = W_n(t) - V_n(t). \quad (5)$$

$W_n(t)$ is positive when fresh buy pressure arrives at level n and negative when buyers withdraw, $V_n(t)$ does the same for sell pressure, and $e_n(t)$ is the net excess of buy over sell pressure at level n . We sum $e_n(t)$ across the lookback window to form a 10-dimensional cumulative OFI vector $x_t = (\sum_{\tau} e_1(\tau), \dots, \sum_{\tau} e_{10}(\tau))$ with one entry per LOB level. This is the input to a multinomial logistic regression classifier

$$\Pr(y_t = c | x_t) = \frac{\exp(\beta_c^\top x_t)}{\sum_{j=0}^2 \exp(\beta_j^\top x_t)}, \quad c \in \{0, 1, 2\}, \quad (6)$$

where the coefficients β_c are fitted by maximising the regularised log-likelihood with class-balanced weights so that no single class dominates training. Equations (3)–(5) are a time-uniform adaptation of the per-event definition in [Cont et al. \(2014\)](#), aggregating flow across intra-snapshot events.

4.2. Plain LSTM

The plain LSTM baseline takes the raw 50×40 LOB sequence directly and processes it with a single Long Short-Term Memory layer of 64 hidden units, followed by a dropout layer (rate 0.2) and a softmax classifier (Table 2). The LSTM gates control which information from the past 12.5 s to retain or discard, allowing the network to capture dependencies across the lookback window. No convolutional preprocessing is applied, so the LSTM must learn spatial relationships among the 40 LOB features and temporal dynamics simultaneously through its recurrent state updates. By comparing plain LSTM with DeepLOB, LiT and Attentive DeepLOB we can assess whether the specialised front ends provide a meaningful benefit over a model that must discover spatial structure purely through recurrence.

Table 2. Plain LSTM architecture. Input shape (50, 40). 27,075 parameters.

Layer	Output	Purpose
Input	50×40 matrix	Raw LOB window
LSTM (64 units)	64-dim vector	Joint spatial-temporal
Dropout (0.2)	64-dim vector	Regularisation
Softmax (3)	3-dim probability	Class probabilities

4.3. DeepLOB

DeepLOB ([Zhang et al., 2019](#)) is a hybrid CNN-LSTM architecture purpose-built for LOB data. Its three-block

Table 3. DeepLOB architecture. Input shape (50, 40, 1). 61,907 parameters.

Layer	Output	Purpose
Input	$50 \times 40 \times 1$	Raw LOB window
Conv. Block 1	$50 \times 20 \times 16$	Price-volume pairing
Conv. Block 2	$50 \times 10 \times 16$	Cross-level aggregation
Conv. Block 3	$50 \times 1 \times 16$	Level compression
Inception	$50 \times 1 \times 96$	Multi-timescale features
LSTM (64)	64-dim vector	Long-range temporal
Softmax (3)	3-dim probability	Class probabilities

convolutional front end encodes the known spatial structure of the LOB snapshot before passing a compact feature representation to an LSTM for temporal modelling. The key insight is that the LOB has a two-dimensional structure of levels and features, and convolutional filters with appropriate strides can capture price-volume pairing and cross-level interactions without needing to learn from scratch.

Convolutional blocks. Convolution Block 1 applies a (1×2) filter with stride (1, 2) and 16 output channels operating on the $(T, 40, 1)$ input. The kernel width of two with stride two ensures that each filter sees exactly one (price, volume) pair at a single LOB level on a single side of the book, so that the first layer learns a local price-volume coupling rather than mixing features across levels or sides. The output shape is $(T, 20, 16)$, where the 20 spatial positions correspond to the 10 bid and 10 ask levels. Each strided convolution is followed by two additional (4×1) convolution layers that smooth the features along the time axis, a Leaky ReLU activation with $\alpha = 0.01$ and batch normalisation. Convolution Block 2 applies another (1×2) strided filter, reducing to $(T, 10, 16)$ and aggregating information across adjacent levels on the same side of the book. Convolution Block 3 applies a (1×10) filter that compresses all 10 remaining spatial positions into a single feature vector per timestep, followed by two (4×1) temporal smoothing convolutions as in the preceding blocks, yielding a $(T, 1, 16)$ tensor with no spatial dimension remaining.

Inception module and head. The Inception module ([Szegedy et al., 2015](#)) runs three parallel branches on the compressed temporal features and concatenates them, simultaneously encoding temporal patterns at multiple scales. The first branch is a 3×1 convolution preceded by a 1×1 bottleneck, capturing patterns spanning three timesteps (~ 750 ms). The second is a 5×1 convolution capturing ~ 1.25 s patterns. The third is a max-pool branch that isolates the dominant feature activation. The concatenated $(T, 1, 96)$ output is read by a 64-unit LSTM whose final hidden state feeds a softmax classifier (see Table 3). The model has 61,907 parameters. We train with Adam at $\text{lr} = 10^{-3}$, $\epsilon = 10^{-7}$, departing from the original paper’s $\text{lr} = 10^{-2}$, $\epsilon = 1$, which we found produced validation-loss oscilla-

tion on our more noisy 250 ms BTC data. The original setting was tuned for the FI-2010 dataset, where order book updates are sampled at intervals of ten LOB events on lower-volatility Finnish equities. On uniform-time crypto data the per-step gradient signal is noisier and the more conservative learning rate is required to keep training stable.

4.4. LiT - Limit Order Book Transformer

LiT (Xiao et al., 2025) replaces the convolutional front end of DeepLOB with a transformer-based self-attention mechanism, removing the locality constraints present in convolutional filters.

Input representation. The 50×40 LOB window is reshaped into a 3D tensor of shape $(H \times T \times 2) = (20 \times 50 \times 2)$, where $H = 2 \times 10$ is twice the number of LOB levels with the two channels representing price and volume.

Structured patches. The tensor is divided into non-overlapping rectangular patches of size $P_h \times P_w = 10 \times 2$. With $P_h = H/2 = 10$, each patch covers all 10 levels on one side of the book over a 500 ms temporal window, grouping the price and volume entries on a single side together rather than interleaving them. This design ensures that patches contain coherent market structure. The total number of patches per sample is $(H/P_h) \times (T/P_w) = 2 \times 25 = 50$. Each flattened 40-dimensional patch is projected by a shared linear layer to a $d_{\text{model}} = 64$ embedding and a learnable positional embedding is added.

Transformer blocks and head. Two stacked Transformer blocks each apply multi-head self-attention ($n_{\text{heads}} = 4$, $d_k = 16$, feed-forward hidden dimension 128) with layer normalisation and residual connections. Self-attention allows every patch to attend to every other patch with no locality constraint, so a bid-side patch at time t can directly attend to an ask-side patch at time $t - 10$, capturing cross-side and cross-time correlations invisible to a convolutional filter. After the transformer blocks, a 64-unit LSTM processes the sequence of patch representations, and the final hidden state feeds a 32-unit dense layer with ReLU, a dropout layer and a softmax classifier (Table 4). LiT has 105,347 parameters.

Table 4. LiT architecture. Input shape (20, 50, 2). 105,347 parameters.

Layer	Output	Purpose
Input	$20 \times 50 \times 2$	Price, volume channels
Patch embedding	50×64	Patches + pos. embed.
Transformer $\times 2$	50×64	Global self-attention
LSTM (64)	64-dim vector	Sequential memory
Dense (32), ReLU	32-dim vector	Nonlinear readout
Dropout (0.1)	32-dim vector	Regularisation
Softmax (3)	3-dim probability	Class probabilities

4.5. Attentive DeepLOB: Our Proposed Extension

Comparing the four established baselines reveals a clean trade off. DeepLOB’s strided convolutional front end captures local price-volume and cross level interactions effectively and provides a strong domain-specific inductive bias, but its receptive field is fundamentally local. LiT’s transformer captures longer-range dependencies between non-adjacent regions of the order book but discards the explicit price-volume pairing that makes DeepLOB so well suited to LOB data. Drawing on our understanding of LOB dynamics and on standard deep learning practice, we propose Attentive DeepLOB, which retains DeepLOB’s convolutional and Inception front end and inserts two axial self-attention blocks between the Inception output and the LSTM.

Unlike the original DeepLOB, all convolutions in Attentive DeepLOB use `padding='same'`, so the level dimension is preserved throughout. After the Inception block and a 1×1 projection to $d_{\text{model}} = 32$ channels the tensor has shape $(T, 10, 32)$. After the two axial attention blocks it is reshaped to $(T, 320)$ before being passed to the LSTM.

Axial attention factorises 2D self-attention into two cheaper 1D attention operations applied along each axis, reducing the computational cost from quadratic in the total number of positions to linear in the product of axis lengths. We apply attention along the level axis first, capturing cross-level dependencies such as deep-book liquidity affecting closer mid-price dynamics, and then along the time axis, capturing long-range temporal dependencies the LSTM may compress away. Each axial block uses 4 heads, $d_{\text{model}} = 32$, $d_k = 8$, dropout 0.1, and a residual connection with layer normalisation. The output is reshaped, passed to a 64-unit LSTM and softmax head, giving 130,931 total parameters.

Table 5. Attentive DeepLOB architecture. Input shape (50, 40, 1). 130,931 parameters.

Layer	Output	Purpose
Input	$50 \times 40 \times 1$	Raw LOB window
Conv. Block 1	$50 \times 20 \times 16$	Price-volume pair
Conv. Block 2	$50 \times 10 \times 16$	Cross-level aggr.
Conv. Block 3	$50 \times 10 \times 16$	Level features
Inception	$50 \times 10 \times 96$	Multi-timescale feats
Convolution 1×1	$50 \times 10 \times 32$	Channel reduction
Axial-level attn.	$50 \times 10 \times 32$	Cross-level deps.
Axial-time attn.	$50 \times 10 \times 32$	Temporal deps.
Reshape	50×320	Flatten for LSTM
LSTM (64)	64-dim vector	Long-range temp.
Softmax (3)	3-dim probability	Class probabilities

5. Training

Shared protocol. All deep learning models are trained with the Adam optimiser (Kingma & Ba, 2015) using categorical cross-entropy loss and mini-batches of 256. We use valida-

tion loss for both early stopping and learning rate reduction, with class-balanced sample weights throughout to prevent the optimiser from collapsing onto the majority training class. Validation loss is preferred over validation accuracy as the stopping criterion because cross-entropy loss is the actual quantity being optimised, and tracking it directly avoids the situation where accuracy improves marginally while the model becomes less calibrated (Guo et al., 2017).

Learning rates are tuned per architecture based on the optimiser scale that the architecture’s normalisation layers expect, with 5×10^{-4} for the Plain LSTM, 1×10^{-3} with $\epsilon = 10^{-7}$ for DeepLOB, 3×10^{-4} for LiT (a smaller value is appropriate for transformer-style models with layer normalisation, following standard practice), and 5×10^{-4} for Attentive DeepLOB. We apply early stopping with a patience parameter of 8 epochs on validation loss and restore the best-weight checkpoint, and reduce the learning rate by a factor of 0.5 on validation loss plateaus with a patience of 5 epochs, down to a minimum of 10^{-6} . Training runs for at most 50 epochs (our runs very rarely reached this). We train one independent model per horizon for each architecture, so that the filters and attention weights of each instance specialise to the temporal scale of the prediction task.

6. Numerical Results

Evaluation protocol and metric choice. We report four metrics on the held-out test set of approximately 344,563 samples per horizon (14 January 2023). *Accuracy* gives the overall fraction of correctly classified samples and is the most intuitive summary, although it is misleading under class imbalance because it can be high simply by predicting the majority class. *Balanced accuracy* is the unweighted average of per-class recall and is therefore suited to dealing with imbalance, treating an error on the minority Stationary class at 5 s as just as costly as an error on the majority Down or Up classes. *Weighted F1* averages per-class F1 scores using class support as weights and reflects practical performance on the realised class mix in the test set. *Macro F1* averages per-class F1 unweighted and is therefore the most sensitive to minority class performance, penalising any model that ignores or systematically misclassifies one of the directional classes. Reporting all four answers different questions and lets the reader see which model wins under which evaluation criterion.

6.1. Main results

Table 6 (See Appendix) compares all five models across horizons. Figure 6 (See Appendix) plots accuracy and macro F1 as a function of horizon for all models, and Figure 7 (See Appendix) shows per-class F1 scores. Normalised confusion matrices are provided in Figures 4 and 5 (See Appendix).

OFI baseline. The OFI logistic model achieves accuracy in the 40-43% range across the three horizons, with balanced accuracy around 44-49%. Despite its simplicity it performs reasonably well compared to the 33% chance level with our balanced labels. The deep sequence models substantially outperform OFI across all horizons and metrics, with the smallest gap at 1 s where Plain LSTM exceeds OFI by 21 percentage points of accuracy and the largest at 5 s where Attentive DeepLOB exceeds OFI by 26 percentage points. The Plain LSTM achieves 63.9% accuracy at the 1 s horizon, demonstrating that even without a convolutional front end a recurrent network extracts substantial signal from the raw 40-feature LOB window.

Attentive DeepLOB versus DeepLOB. Attentive DeepLOB outperforms the original DeepLOB at the 1 s horizon across all four metrics (68.5% vs. 64.1% accuracy, 68.5% vs. 64.0% macro F1) and at 5 s (66.2% vs. 65.3% accuracy). At 2 s, however, DeepLOB leads (69.7% vs. 69.0% accuracy, 67.2% vs. 65.3% macro F1). At 1 s the prediction target is dominated by microstructure noise and the cross-level signal that the axial attention surfaces is most informative, while at the 2 s horizon the prediction target is cleaner and the more rigid convolutional bias of plain DeepLOB fits it well on its own.

LiT. LiT is the top performer on balanced accuracy at 2 s (69.4%) and achieves competitive results at 1 s (66.1% accuracy). Its patch-based transformer captures longer-range dependencies without locality constraints, but its lack of explicit price-volume pairing in the patch embedding may limit its advantage over Attentive DeepLOB, which retains the strided convolutional front end.

Performance versus horizon. OFI accuracy falls from 43% to 40% as the horizon lengthens, because the stationary fraction collapses and the linear model cannot exploit more complex directional patterns. The deep models broadly improve from 1 s to 2 s and then slightly decline or plateau at 5 s, with DeepLOB achieving its peak accuracy at 2 s.

Per-class behaviour. The per-class results in Section C (See Appendix) reveal an important asymmetry at 1 s. OFI achieves reasonable precision on the directional classes (0.52/0.52) but low recall (0.36/0.40), whereas the deep models achieve more balanced recall across classes. At 5 s the stationary class collapses to only 4.3% of the test set, making balanced accuracy and macro F1 the most informative aggregate metrics at that horizon.

Overfitting diagnostics. Table 7 (See Appendix) shows that the train-validation accuracy gap is moderate (4-7 percentage points) and consistent across architectures on this dataset, in contrast to the larger gap that was reported for DeepLOB in the original paper. This suggests that on the 250 ms BTC sample the day based split and class-balanced

training together provide substantial regularisation. Attentive DeepLOB’s larger parameter count (131K vs. 62K for DeepLOB) does not result in a larger gap.

6.2. Statistical significance

With approximately 344,500 paired test predictions per horizon, standard errors on individual accuracy estimates are very small. We use McNemar’s test with continuity correction on the 2×2 contingency table of agreement and disagreement between two models on each test point to assess pairwise differences, since this paired test exploits the fact that all models score the same test set. Table 11 (See Appendix) reports key pairwise comparisons. Attentive DeepLOB significantly outperforms DeepLOB at 1 s ($\chi^2 = 4140$, $p < 10^{-300}$, +4.38 pp) and at 5 s ($p = 8.2 \times 10^{-31}$, +0.86 pp), while DeepLOB outperforms Attentive DeepLOB at 2 s ($p = 3.3 \times 10^{-32}$, +0.74 pp). Attentive DeepLOB also significantly outperforms LiT at all three horizons ($p < 10^{-150}$ in every case).

The one notable null result is DeepLOB versus Plain LSTM at 1 s, where the +0.18 pp gap reaches only $p = 0.013$ and the 60,000+ paired disagreements show the two models making substantially different errors of comparable aggregate magnitude. This suggests that DeepLOB’s convolutional front end provides little benefit over a plain recurrent baseline at the shortest horizon. Attentive DeepLOB’s significant 1 s advantage over both DeepLOB and Plain LSTM therefore appears to be driven primarily by the axial attention component rather than the inherited convolutional front end. At our test set sizes, sub-percentage-point differences between deep models are statistically distinguishable but should not be over-interpreted as economically material once transaction costs and market impact are accounted for.

6.3. Adaptation via fine-tuning

Following the protocol of Xiao et al. (2025) we test how well each architecture’s representations transfer to later market conditions. The test day (14 January) is split in half, with the first half becoming the fine-tuning adaptation set and the second half the fine-tuning evaluation set ($\approx 172,255$ sequences each). We compare two strategies. Under *zero-shot* the pre-trained model is applied with no adaptation, and under *fine-tuned* all layers except the final two dense layers are frozen and those layers are updated on the adaptation data at a reduced learning rate. Results are reported for 1 s.

Table 9 shows that fine-tuning consistently improves all four deep models. The largest absolute gain goes to DeepLOB (from 63.9% to 69.0% accuracy), followed by LiT (65.7% to 70.0%). Attentive DeepLOB starts from the strongest zero-shot position (68.6%) and reaches 70.6% after fine-tuning. The consistent gains confirm that the first half of the test day contains distributional information not present in

the training window. Fine-tuning results are illustrated in Figure 8 (See Appendix).

6.4. Attention and saliency analysis

To probe what each model learns we extract gradient $\times$ input saliency maps for DeepLOB and Attentive DeepLOB, and self-attention weights for LiT, over 200 windows drawn from the fine-tuning evaluation set at the 1 s horizon (results shown in Appendix Figures 9 to 11).

Recency bias. Both DeepLOB and Attentive DeepLOB display a strong recency bias in their saliency maps. The 10 most recent timesteps (2.5 s) account for saliency roughly $8.8\times$ and $9.0\times$ higher than the 10 most distant timesteps respectively. This suggests the LSTM compresses older history and that the Inception module’s multi-scale features mostly use recent book dynamics. While this is expected for short-term prediction, it raises the question of whether reducing the lookback window would be more efficient.

Attentive DeepLOB axial attention. The axial level attention in Attentive DeepLOB is broadly uniform across levels, with level 3 receiving the highest attention (0.105) and level 8 the lowest (0.095). The axial time attention shows essentially no recency bias (ratio $1.01\times$) in contrast to the saliency map. This pattern implies that the axial attention path is learning long range temporal features that are complementary to those captured by the convolutional and LSTM path, rather than encoding the same recency-dominated information. The uniform level attention could suggest the attention mechanism is not recovering the well established microstructure finding that top of book levels are the most informative. However, since the axial attention blocks operate on features that have already been processed by the strided convolutional front end, it is also plausible that level importance has already been encoded earlier in the network.

7. Limitations and Future Research

Our study has several limitations that affect how the results should be interpreted. We train on a five day window that captures only a single market regime. Ideally, models would be evaluated on rolling multi week samples spanning bull and bear trends, volatility shocks and liquidity events, none of which are represented in our narrow January 2023 window. The evaluation is restricted to a single asset and exchange. Limit order book dynamics differ substantially across instruments, venues and market conditions, so the relative ranking of architectures, in particular the 1 s advantage of Attentive DeepLOB, may not generalise beyond this specific setting. A deeper practical limitation is that classification accuracy and F1 are necessary but insufficient criteria for deployment. A model that correctly predicts

Table 6. Test-set performance for all five models across prediction horizons. Best result per horizon and metric is bold. Bal. Acc. = balanced accuracy, F1-W = weighted F1, F1-M = macro F1.

Horizon	Model	Acc.	Bal. Acc.	F1-W	F1-M
1 s	OFI Logistic	0.430	0.444	0.433	0.431
	Plain LSTM	0.639	0.658	0.638	0.638
	LiT	0.661	0.679	0.660	0.661
	DeepLOB	0.641	0.665	0.639	0.640
	Attentive DeepLOB	0.685	0.699	0.683	0.685
2 s	OFI Logistic	0.410	0.464	0.441	0.403
	Plain LSTM	0.584	0.652	0.605	0.569
	LiT	0.646	0.694	0.656	0.627
	DeepLOB	0.697	0.687	0.700	0.672
	Attentive DeepLOB	0.690	0.654	0.690	0.653
5 s	OFI Logistic	0.401	0.492	0.474	0.368
	Plain LSTM	0.602	0.627	0.629	0.515
	LiT	0.633	0.657	0.654	0.547
	DeepLOB	0.653	0.649	0.664	0.570
	Attentive DeepLOB	0.662	0.643	0.671	0.577

the direction of a 0.1 USDT mid-price move provides no trading edge once the bid-ask spread, exchange fees and market impact of order placement are accounted for. Finally, our smoothed labelling scheme calibrates α on the training slice, following Zhang et al. (2019). This is consistent across all five models compared here, but means the labels are operational definitions rather than universal truths, and α targets calibrated on one dataset are not directly transferable to another. Accuracy figures should therefore be compared across studies with care. Future work should address these limitations by training on longer and more diverse windows, exploring hybrid architectures that apply patch based attention over convolutional level features rather than raw input patches, and evaluating models within the LOBFrame framework of Briola et al. (2025), which translates classification performance into the probability of profitably entering and exiting a trade.

8. Conclusion

We have presented a comparative study of five approaches to short horizon mid-price prediction on a five-day sample of Bitcoin perpetual futures LOB data (10-14 January 2023), applying a strict day based train, validation and test split that yields approximately 1,034K training, 345K validation and 345K test sequences. The OFI baseline lies in the 40-43% accuracy range against a 33% chance level. All deep sequence models substantially outperform OFI, with Attentive DeepLOB achieving the strongest overall accuracy at 1 s (68.5%) and 5 s (66.2%), and DeepLOB leading at 2 s (69.7%). Across horizons, Attentive DeepLOB obtains the highest accuracy on two of three horizons and the highest macro F1 on two of three, suggesting that combining convolutional spatial inductive bias with axial self-attention is a small but consistent improvement over either

component alone on this dataset. These figures are lower than the headline numbers in the original papers. Zhang et al. (2019) report 84.5% accuracy for DeepLOB on the pre-normalised FI-2010 benchmark and Xiao et al. (2025) report up to 68.3% for LiT on event-sampled Binance data, but this gap is itself a central finding of this work. Both prior evaluations rely on sampling that concentrates observations on moments of market activity, making the prediction structurally easier. The fact that our models achieve competitive performance under the stricter uniform clock-time setting, and that Attentive DeepLOB matches LiT’s best reported accuracy despite the harder evaluation conditions, suggests these architectures do generalise beyond their original benchmarks, though with the more modest figures one would expect when models are tested under more realistic trading conditions.

One finding we did not expect is the strength and consistency of the fine-tuning gains. Adapting only the final dense layers on the first half of the test day recovers 3-6 percentage points of accuracy for every model, reaching up to 70.6% for fine-tuned Attentive DeepLOB at 1 s. This confirms that even a single day of distribution shift is significant, and that pre trained representations transfer well enough to make periodic fine-tuning a viable strategy.

The axial attention analysis of Attentive DeepLOB reveals a complementary pattern. The gradient $\times$ input saliency path shows a strong recency bias (ratio $\approx 9\times$), while the time-axis attention shows essentially no recency bias (ratio $\approx 1\times$). This suggests axial attention is learning qualitatively different features from the convolutional LSTM path, which may explain the 1 s performance gain.

References

- Berti, L. and Kasneci, G. TLOB: A novel transformer model with dual attention for stock price trend prediction with limit order book data. *arXiv preprint arXiv:2502.15757*, 2025.
- Briola, A., Bartolucci, S., and Aste, T. Deep limit order book forecasting: A microstructural guide. *Quantitative Finance*, 2025. doi: 10.1080/14697688.2025.2522911.
- Cont, R., Kukanov, A., and Stoikov, S. The price impact of order book events. *Journal of Financial Econometrics*, 12(1):47–88, 2014. doi: 10.1093/jfinec/nbt003.
- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pp. 1321–1330, 2017.
- Kercheval, A. N. and Zhang, Y. Modelling high-frequency limit order book dynamics with support vector machines. *Quantitative Finance*, 15(8):1315–1329, 2015. doi: 10.1080/14697688.2015.1032546.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.
- Kisiel, D. and Gorse, D. Axial-LOB: High-frequency trading with axial attention. In *Proceedings of the 2022 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 1327–1333, 2022. doi: 10.1109/SSCI51031.2022.10022284.
- Ntakaris, A., Magris, M., Kannianen, J., Gabbouj, M., and Iosifidis, A. Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods. *Journal of Forecasting*, 37(8):852–866, 2018. doi: 10.1002/for.2543.
- Sirignano, J. and Cont, R. Universal features of price formation in financial markets: Perspectives from deep learning. *Quantitative Finance*, 19(9):1449–1459, 2019. doi: 10.1080/14697688.2019.1622295.
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., and Rabinovich, A. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1–9, 2015. doi: 10.1109/CVPR.2015.7298594.
- Tsantekidis, A., Passalis, N., Tefas, A., Kannianen, J., Gabbouj, M., and Iosifidis, A. Forecasting stock prices from the limit order book using convolutional neural networks. In *Proceedings of the IEEE 19th Conference on Business Informatics*, pp. 7–12, 2017. doi: 10.1109/CBI.2017.23.
- Wang, H. Exploring microstructural dynamics in cryptocurrency limit order books: Better inputs matter more than stacking another hidden layer. *arXiv preprint arXiv:2506.05764*, 2025.
- Xiao, Y., Ventre, C., Wang, Y., Li, H., Huan, Y., and Liu, B. LiT: Limit order book transformer. *Frontiers in Artificial Intelligence*, 8:1616485, 2025. doi: 10.3389/frai.2025.1616485.
- Zhang, Z., Zohren, S., and Roberts, S. DeepLOB: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, 67(11):3001–3012, 2019. doi: 10.1109/TSP.2019.2907260.

A. Data Visualisation

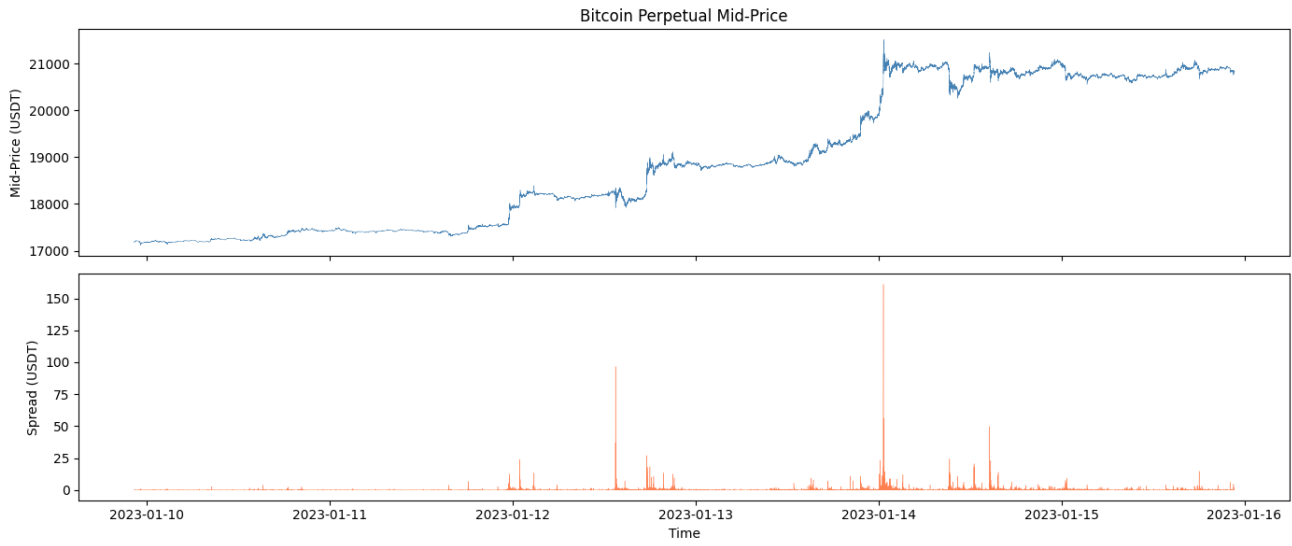


Figure 1. Bitcoin perpetual futures mid-price (top) and bid-ask spread (bottom) over the raw loaded sample (9-15 January 2023, 250 ms intervals). The spread holds at the 0.1 USDT minimum tick for the vast majority of the sample.

B. Overfitting Diagnostics

Table 7. Overfitting diagnostics at the best validation epoch.

Horizon	Model	Ep.	Tr. Acc	Val Acc
1 s	Plain LSTM	16	0.806	0.762
	DeepLOB	2	0.815	0.779
	LiT	14	0.818	0.777
	Attentive DeepLOB	3	0.821	0.780
2 s	Plain LSTM	7	0.798	0.739
	DeepLOB	2	0.814	0.762
	LiT	10	0.815	0.757
	Attentive DeepLOB	2	0.810	0.758
5 s	Plain LSTM	4	0.776	0.709
	DeepLOB	2	0.791	0.729
	LiT	4	0.783	0.721
	Attentive DeepLOB	1	0.762	0.719

C. Detailed Per-Class Results

Model	Class	$k = 4$ (1 s)			$k = 8$ (2 s)			$k = 20$ (5 s)		
		Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1
OFI	Down	0.525	0.363	0.429	0.608	0.361	0.453	0.660	0.373	0.477
	Stationary	0.324	0.574	0.414	0.177	0.640	0.278	0.069	0.701	0.125
	Up	0.523	0.395	0.450	0.608	0.392	0.477	0.667	0.402	0.502
Plain LSTM	Down	0.769	0.506	0.610	0.755	0.533	0.625	0.722	0.534	0.614
	Stationary	0.513	0.843	0.638	0.306	0.872	0.452	0.157	0.686	0.256
	Up	0.710	0.626	0.665	0.731	0.551	0.629	0.692	0.661	0.676
LiT	Down	0.755	0.591	0.663	0.729	0.652	0.688	0.736	0.582	0.650
	Stationary	0.549	0.849	0.667	0.386	0.851	0.531	0.191	0.715	0.302
	Up	0.722	0.596	0.653	0.768	0.580	0.661	0.704	0.676	0.689
DeepLOB	Down	0.779	0.541	0.638	0.737	0.693	0.714	0.732	0.602	0.661
	Stationary	0.513	0.895	0.652	0.523	0.656	0.582	0.246	0.642	0.356
	Up	0.724	0.559	0.631	0.724	0.713	0.719	0.684	0.704	0.694
Attentive DeepLOB	Down	0.767	0.585	0.664	0.741	0.662	0.699	0.706	0.666	0.685
	Stationary	0.598	0.839	0.698	0.532	0.540	0.536	0.258	0.599	0.361
	Up	0.710	0.673	0.691	0.691	0.762	0.725	0.707	0.664	0.685

Table 8. Per-class precision, recall, and F1 for all five models across all three forecast horizons.

D. Fine-Tuning Results

Table 9. Fine-tuning results at 1 s horizon. Zero-shot applies the pre-trained model directly. Fine-tuned freezes all but the final dense layers.

Model	Setting	Accuracy	Macro F1
Plain LSTM	Zero-shot	0.635	0.629
	Fine-tuned	0.670	0.669
DeepLOB	Zero-shot	0.639	0.631
	Fine-tuned	0.690	0.690
LiT	Zero-shot	0.657	0.652
	Fine-tuned	0.700	0.699
Attentive DeepLOB	Zero-shot	0.686	0.683
	Fine-tuned	0.706	0.706

E. Training Curves and Confusion Matrices

Training & Validation Loss

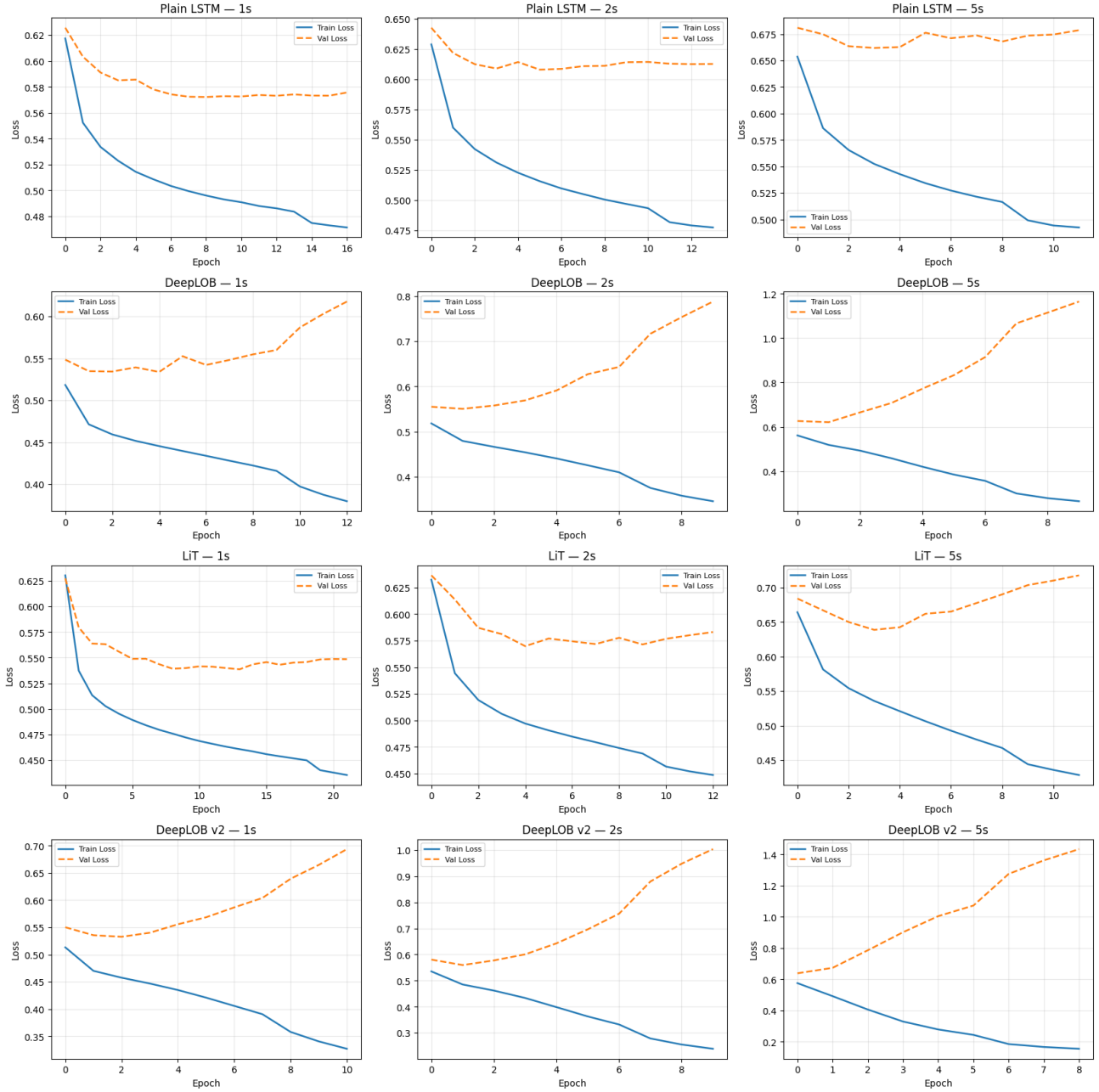


Figure 2. Training (solid) and validation (dashed) loss curves for all four deep models across the three forecast horizons.

Training & Validation Accuracy

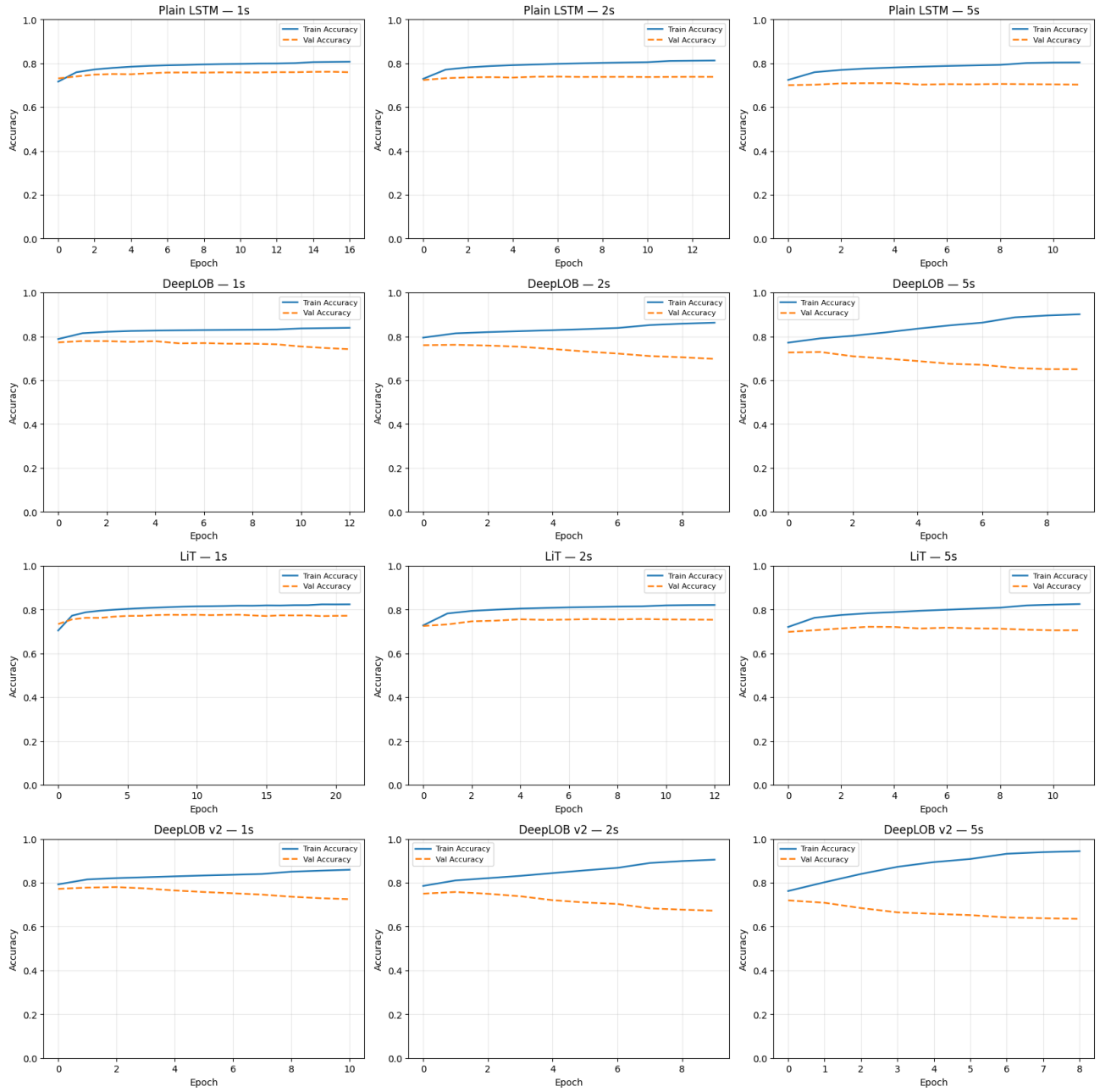


Figure 3. Training (solid) and validation (dashed) accuracy curves for all four deep models across the three forecast horizons.

Normalised Confusion Matrices

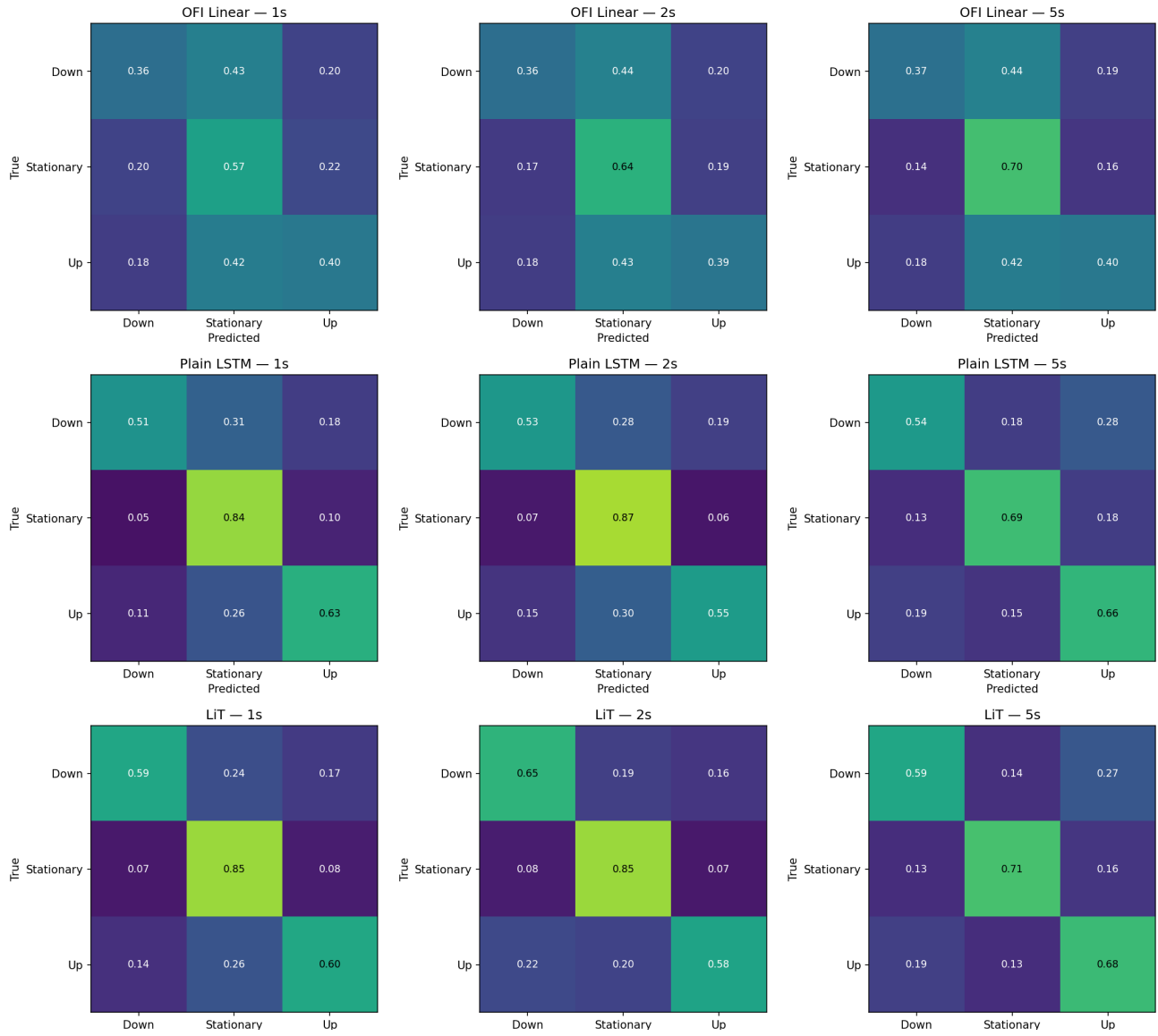


Figure 4. Row-normalised confusion matrices for OFI, Plain LSTM and LiT across all three forecast horizons (continued overleaf).

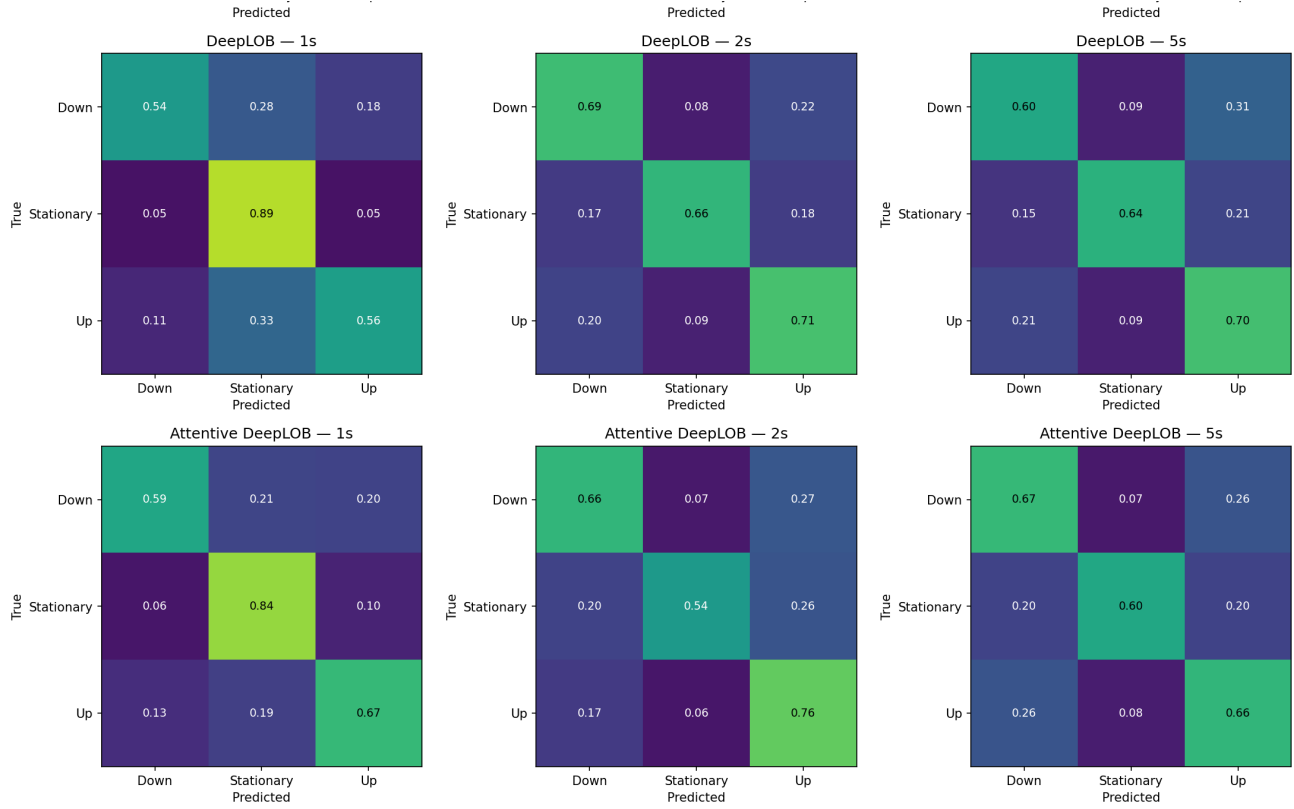


Figure 5. Row-normalised confusion matrices for DeepLOB and Attentive DeepLOB across all three forecast horizons.

F. Comparative Performance and Fine-Tuning Figures

Core Test Performance

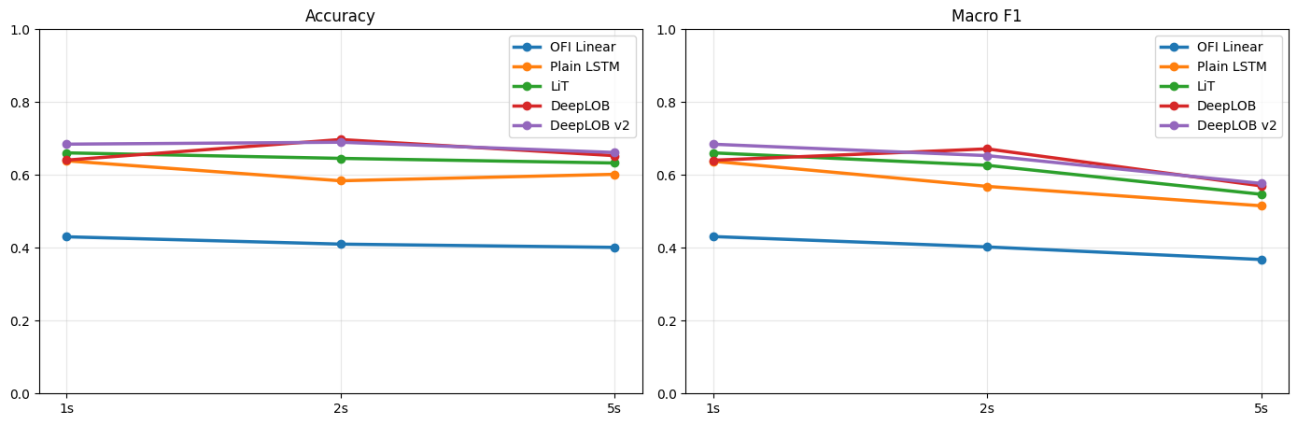


Figure 6. Test-set accuracy (left) and macro F1 (right) for all five models across the three forecast horizons.

Per-Class F1 by Model and Horizon

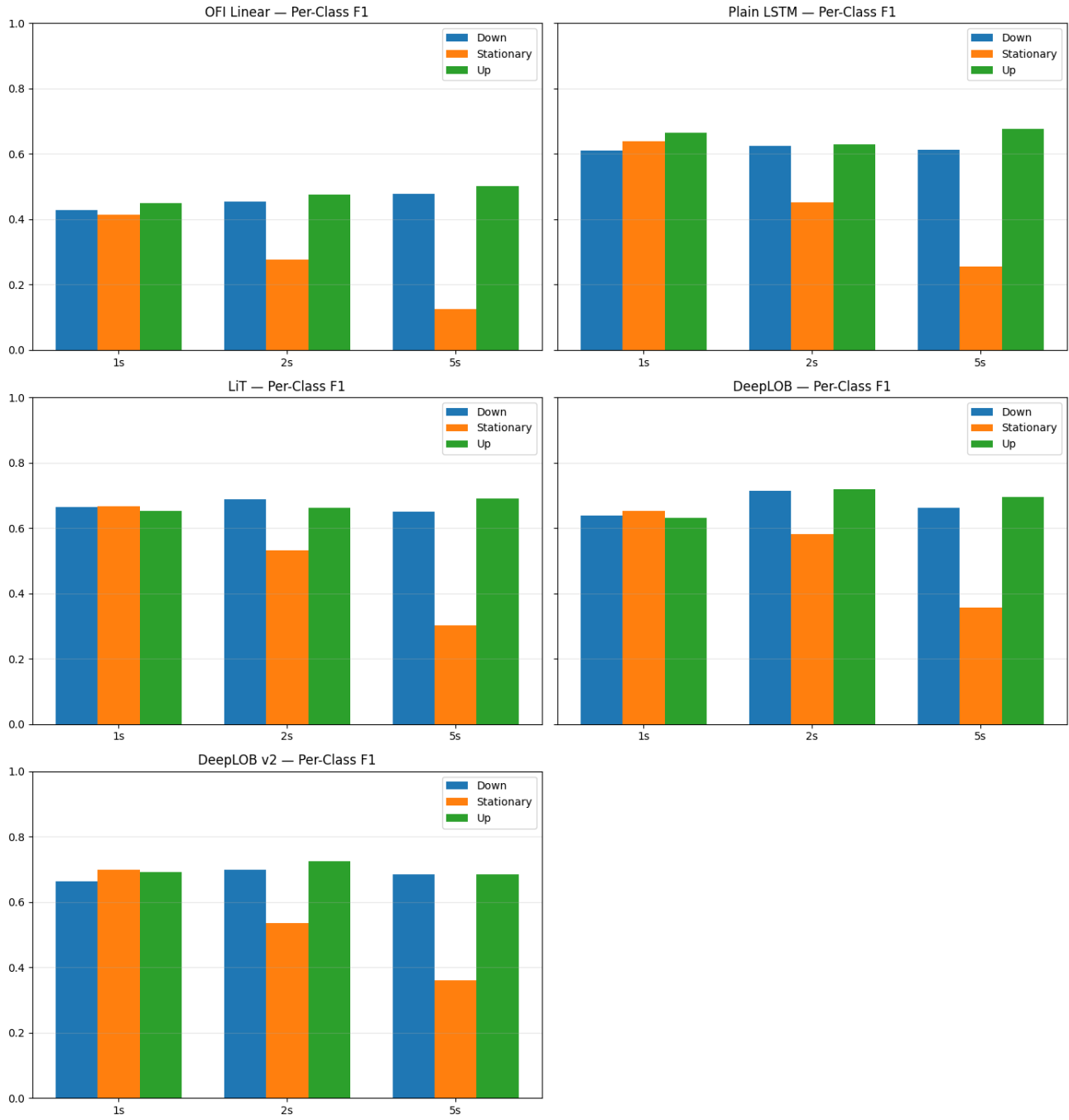


Figure 7. Per-class F1 scores (Down, Stationary, Up) for each model across the three forecast horizons.

Fine-tuning protocol per model — 1s horizon

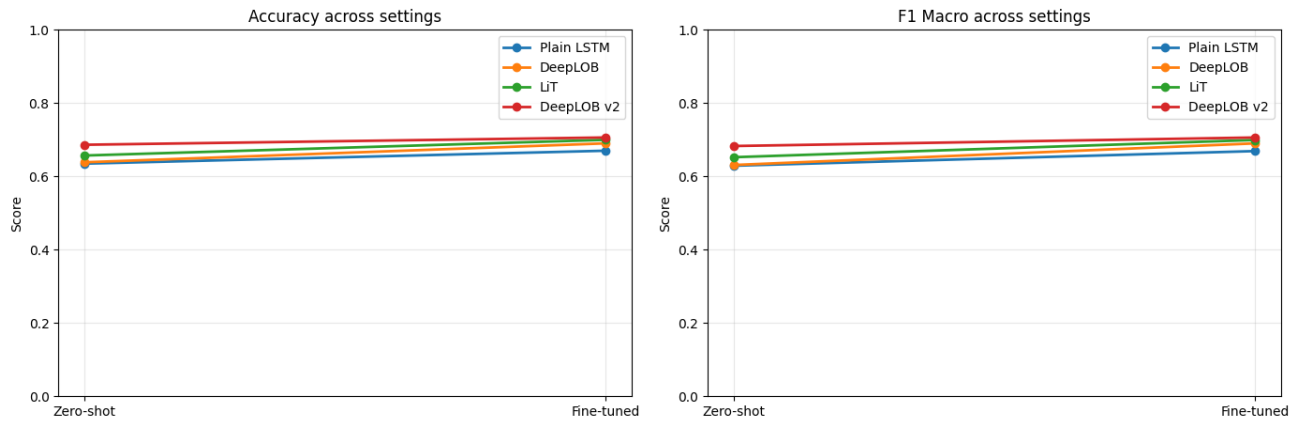


Figure 8. Fine-tuning results at $k = 4$ (1 s). Zero-shot applies the pre-trained model directly. Fine-tuned freezes all but the final classification head.

G. Attention and Saliency Analysis

LiT Self-Attention — 1s horizon

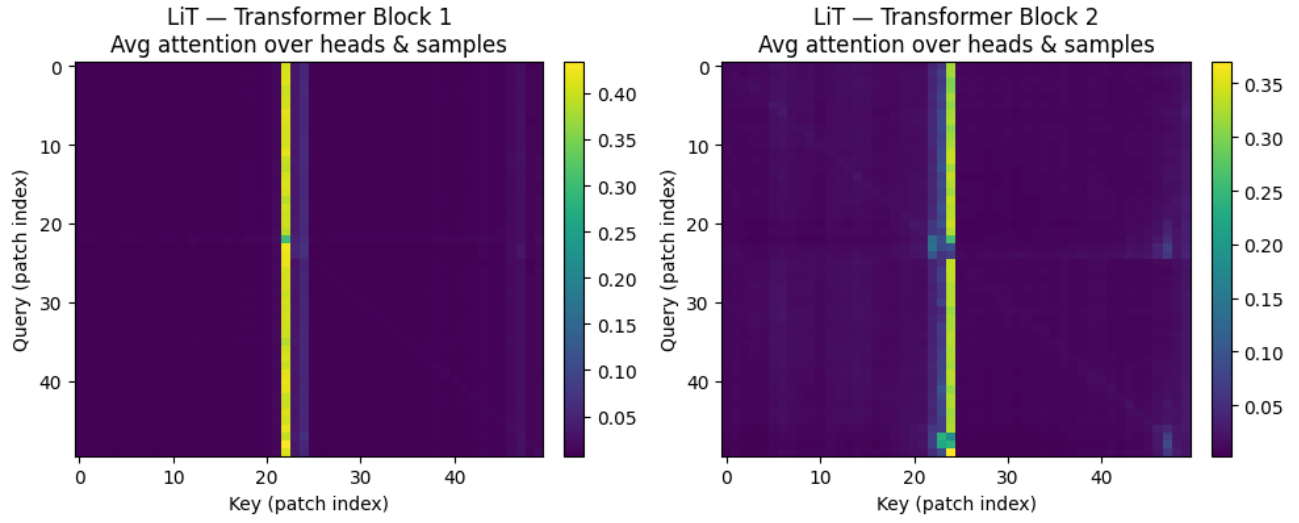


Figure 9. Average self-attention maps for LiT's two transformer blocks at $k = 4$, averaged over 200 test windows (50×50 patches, rows = query, columns = key).

Input saliency comparison — 1s horizon

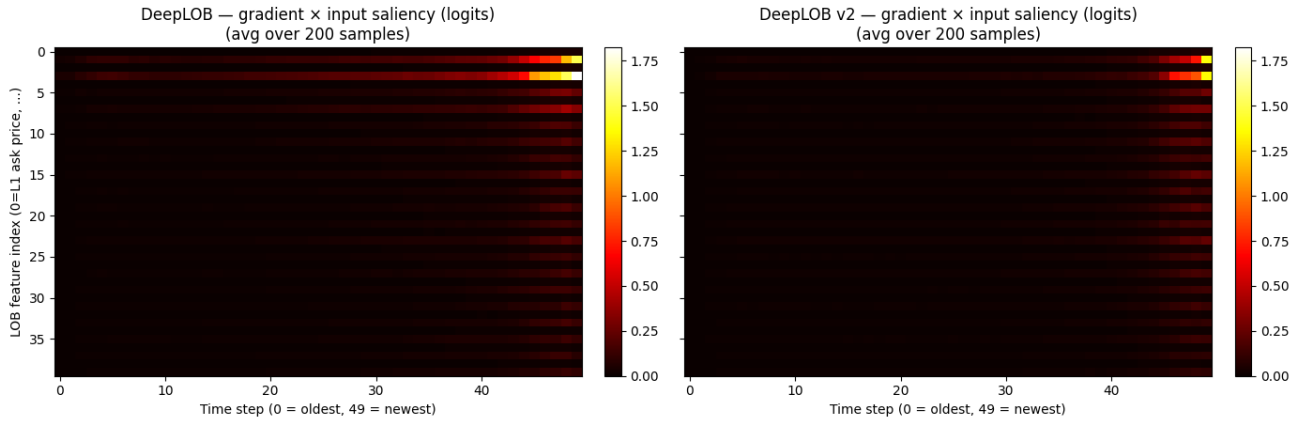


Figure 10. Gradient×input saliency maps for DeepLOB and Attentive DeepLOB at $k = 4$, averaged over 200 test windows. Both models concentrate saliency on the most recent timesteps (recency ratio $\approx 9\times$).

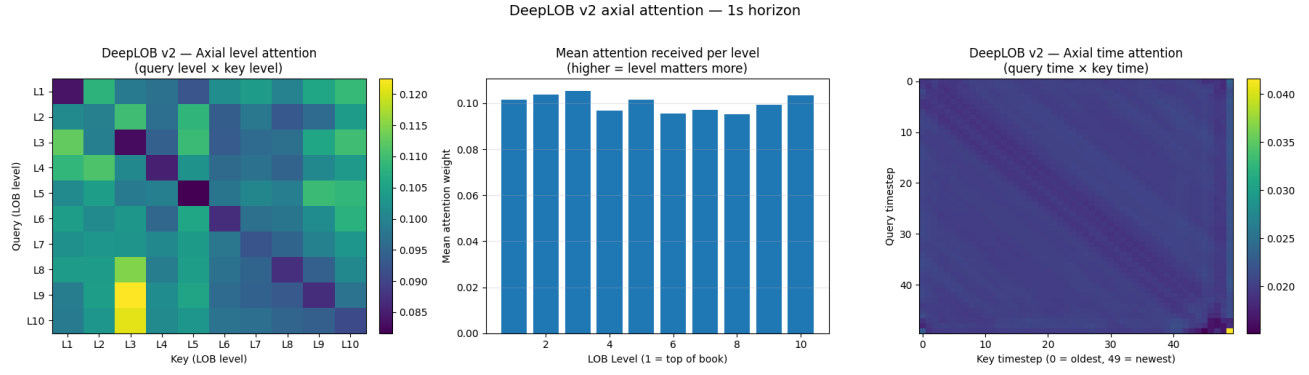


Figure 11. Attentive DeepLOB axial attention at $k = 4$: level-axis attention (near-uniform across 10 LOB levels) and time-axis attention (no recency bias, ratio $\approx 1\times$).

H. Hyperparameter Summary

Table 10. Hyperparameter settings.

Hyperparameter	Value
Lookback window T	50 (12.5 s)
Batch size	256
Optimiser	Adam
LR (Plain LSTM)	5×10^{-4}
LR (DeepLOB)	1×10^{-3} , $\epsilon = 10^{-7}$
LR (LiT)	3×10^{-4}
LR (Attentive DeepLOB)	5×10^{-4}
LR reduction factor / patience	0.5 / 5 epochs
Early stopping patience	8 epochs (val. loss)
Max epochs	50
Leaky-ReLU slope (DeepLOB)	0.01
Horizons k	4, 8, 20 (1 s, 2 s, 5 s)
Split	day based: 3 train / 1 val / 1 test
Train / Val / Test rows	$\approx 1,034\text{K} / 345\text{K} / 345\text{K}$
Train dates	10-12 January 2023
Val date	13 January 2023
Test date	14 January 2023
Total snapshots (5 full days)	$\approx 1,723\text{K}$ (250 ms interval)
Normalisation	Rolling z-score (5-date window)
LiT ($P_h, P_w, d_{\text{model}}$)	(10, 2, 64), 2 blocks, 4 heads
Attentive DeepLOB axial attn	$d_{\text{model}} = 32$, 4 heads, dropout 0.1
Attentive DeepLOB conv padding	same (preserves level dim)
Random seed	42

I. McNemar's Pairwise Tests

Table 11. McNemar's pairwise tests with continuity correction across all three horizons. b = samples where A is correct and B is wrong, c = samples where B is correct and A is wrong, Δ = accuracy difference (pp), p = two-sided p-value.

k	A vs B	Δ (pp)	b	c	χ^2	p
1 s	DLv2 vs DeepLOB	+4.38	35,111	20,004	4140	$< 10^{-300}$
	DLv2 vs LiT	+2.38	30,627	22,415	1271	2.2×10^{-278}
	DLv2 vs LSTM	+4.56	37,420	21,699	4180	$< 10^{-300}$
	DeepLOB vs LSTM	+0.18	30,860	30,246	6.1	0.013
	LiT vs DeepLOB	+2.00	32,671	25,776	813	7.4×10^{-179}
	LiT vs LSTM	+2.18	32,622	25,113	976	2.5×10^{-214}
	LSTM vs OFI	+20.86	112,623	40,737	33,695	$< 10^{-300}$
2 s	DLv2 vs DeepLOB	-0.74	22,038	24,590	140	3.3×10^{-32}
	DLv2 vs LiT	+4.43	45,236	29,965	3101	$< 10^{-300}$
	DLv2 vs LSTM	+10.58	61,622	25,175	15,304	$< 10^{-300}$
	DeepLOB vs LSTM	+11.32	60,605	21,606	18,499	$< 10^{-300}$
	LiT vs DeepLOB	-5.17	23,678	41,501	4873	$< 10^{-300}$
	LiT vs LSTM	+6.15	39,775	18,599	7681	$< 10^{-300}$
	LSTM vs OFI	+17.40	103,132	43,182	24,563	$< 10^{-300}$
5 s	DLv2 vs DeepLOB	+0.86	34,783	31,804	133	8.2×10^{-31}
	DLv2 vs LiT	+2.89	35,535	25,575	1623	$< 10^{-300}$
	DLv2 vs LSTM	+5.99	45,078	24,428	6135	$< 10^{-300}$
	DeepLOB vs LSTM	+5.13	45,736	28,065	4231	$< 10^{-300}$
	LiT vs DeepLOB	-2.03	30,811	37,792	710	1.8×10^{-156}
	LiT vs LSTM	+3.10	33,256	22,566	2047	$< 10^{-300}$
	LSTM vs OFI	+20.06	102,436	33,323	35,183	$< 10^{-300}$